Chapter Six

Run-Time Environment

A program as a source code is merely a collection of text (code, statements etc.) and to make it alive, it requires actions to be performed on the target machine. A program needs memory resources to execute instructions. A program contains names for procedures, identifiers etc., that require mapping with the actual memory location at runtime.

By runtime, we mean a program in execution. Runtime environment is a state of the target machine, which may include software libraries, environment variables, etc., to provide services to the processes running in the system.

Runtime support system is a package, mostly generated with the executable program itself and facilitates the process communication between the process and the runtime environment. It takes care of memory allocation and de-allocation while the program is being executed.

Activation Trees

A program is a sequence of instructions combined into a number of procedures. Instructions in a procedure are executed sequentially. A procedure has a start and an end delimiter and everything inside it is called the body of the procedure. The procedure identifier and the sequence of finite instructions inside it make up the body of the procedure.

The execution of a procedure is called its **activation**. An activation record contains all the necessary information required to call a procedure. An activation record may contain the following units (depending upon the source language used).

Temporaries	Stores temporary and intermediate values of an expression.
Local Data	Stores local data of the called procedure.
Machine Status	Stores machine status such as Registers, Program Counter etc., before the procedure is called.
Control Link	Stores the address of activation record of the caller procedure.
Access Link	Stores the information of data which is outside the local scope.
Actual Parameters	Stores actual parameters, i.e., parameters which are used to send input to the called procedure.
Return Value	Stores return values.

Whenever a procedure is executed, its activation record is stored on the **stack**, also **known as control stack.** When a procedure calls another procedure, the execution of the caller is suspended until the

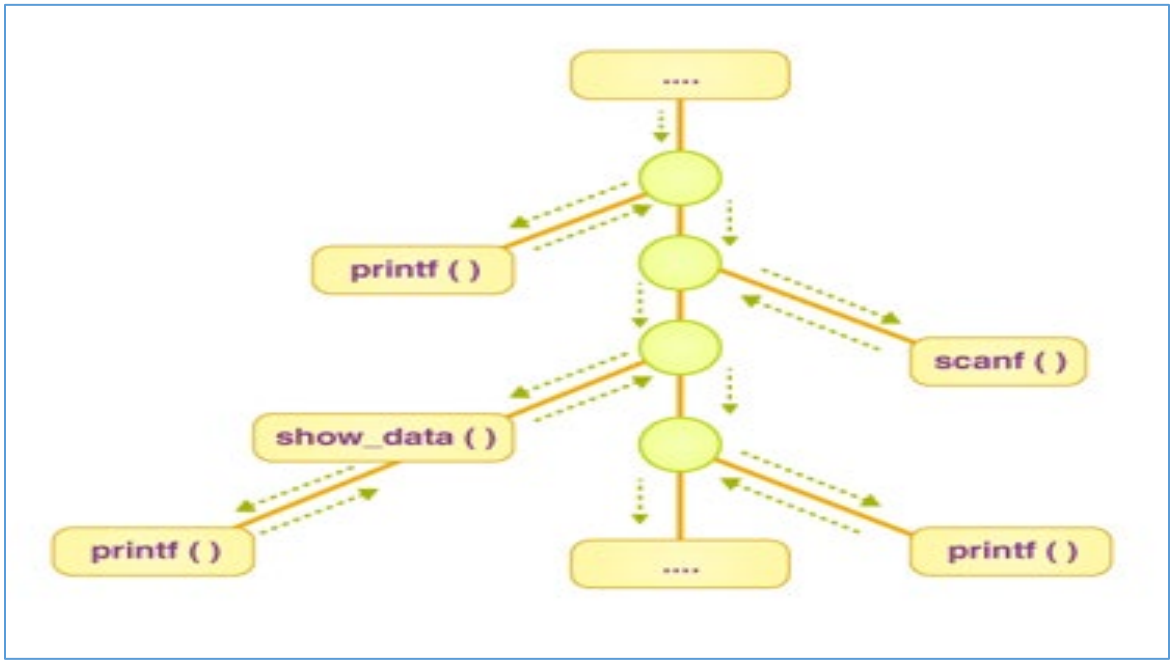
called procedure finishes execution. At this time, the activation record of the called procedure is stored on the stack.

We assume that the program control flows in a sequential manner and when a procedure is called, its control is transferred to the called procedure. When a called procedure is executed, it returns the control back to the caller. This type of control flow makes it easier to represent a series of activations in the form of a tree, known as the **activation tree**.

To understand this concept, we take a piece of code as an example:

```
. . .
printf("Enter Your Name: ");
scanf("%s", username);
show_data(username);
printf("Press any key to continue…");
. . .
int show_data(char *user)
   {
   printf("Your name is %s", username);
   return 0;
   }
. . .
```

Below is the activation tree of the code given.



Now we understand that procedures are executed in depth-first manner, thus stack allocation is the best suitable form of storage for procedure activations.

❖ **Storage Allocation**

Runtime environment manages runtime memory requirements for the following entities:

- **Code**: It is known as the text part of a program that does not change at runtime. Its memory requirements are known at the compile time.
- **Procedures**: Their text part is static but they are called in a random manner. That is why, stack storage is used to manage procedure calls and activations.
- **Variables**: Variables are known at the runtime only, unless they are global or constant. Heap memory allocation scheme is used for managing allocation and de-allocation of memory for variables in runtime.

1. **Static Allocation**

In this allocation scheme, the compilation data is bound to a fixed location in the memory and it does not change when the program executes. As the memory requirement and storage locations are known in advance, runtime support package for memory allocation and de-allocation is not required.

2. **Stack Allocation**

Procedure calls and their activations are managed by means of stack memory allocation. It works in last-in-first-out (LIFO) method and this allocation strategy is very useful for recursive procedure calls.

3. **Heap Allocation**

Variables local to a procedure are allocated and de-allocated only at runtime. Heap allocation is used to dynamically allocate memory to the variables and claim it back when the variables are no more required.

Except statically allocated memory area, both stack and heap memory can grow and shrink dynamically and unexpectedly. Therefore, they cannot be provided with a fixed amount of memory in the system.



As shown in the image above, the text part of the code is allocated a fixed amount of memory. Stack and heap memory are arranged at the extremes of total memory allocated to the program. Both shrink and grow against each other.

Parameter Passing

The communication medium among procedures is known as **parameter passing**. The values of the variables from a calling procedure are transferred to the called procedure by some mechanism. Before moving ahead, first go through some basic terminologies pertaining to the values in a program.

r-value

The **value** of an expression is called its r-value. The value contained in a single variable also becomes an r-value if it appears on the right-hand side of the assignment operator. r-values can always be assigned to some other variable.

l-value

The location of memory (address) where an expression is stored is known as the l-value of that expression. It always appears at the left hand side of an assignment operator.

For example

```
int a = 7; // a is l-value & 7 is r-value
int b = (a + 2); // b is l-value & (a+2) is r-value
int c = (a + b) ;  // c is l-value & (a+b) is r-value
```

NB. l-value is variables whose address is accessible , whereas r-values are not accessible.

Pass by Value

In pass by value mechanism, the calling procedure passes the **r-value** of actual parameters and the compiler puts that into the called procedure's activation record. Formal parameters then hold the values passed by the calling procedure. If the values held by the formal parameters are changed, it should have no impact on the actual parameters.

Pass by Reference

In pass by reference mechanism, the **l-value** of the actual parameter is copied to the activation record of the called procedure. This way, the called procedure now has the address (memory location) of the actual parameter and the formal parameter refers to the same memory location. Therefore, if the value pointed by the formal parameter is changed, the impact should be seen on the actual parameter as they should also point to the same value.

Pass by Copy-restore

This parameter passing mechanism works similar to 'pass-by-reference' except that the changes to actual parameters are made when the called procedure ends. Upon function call, the values of actual parameters are copied in the activation record of the called procedure. Formal parameters if manipulated have no real-time effect on actual parameters (as l-values are passed), but when the called procedure ends, the l-values of formal parameters are copied to the l-values of actual parameters.

Example:

```
int fun( ) {
        int v= 10, u=20;
         int z=add(v,u); //r-value of v and u are passed / actual parameters
        printf (z); //prints
 }
int disp(int x,int y) {   // x and y are  formal parameter
          int s=x+y;
          return s; //  30 will be returned
}
```

When this function ends, the l-value of formal parameter x is copied to the actual parameter y. Even if the value of y is changed before the procedure ends, the l-value of x is copied to the l-value of y making it behave like call by reference.

Symbol Table

Symbol table is an important data structure used in a compiler.

Symbol table is used to store the information about the occurrence of various entities such as objects, classes, variable name, interface, function name etc. it is used by both the analysis and synthesis phases.

The symbol table used for following purposes:

- It is used to store the name of all entities in a structured form at one place.
- It is used to verify if a variable has been declared.
- It is used to determine the scope of a name.
- It is used to implement **type checking** by verifying assignments and expressions in the source code are semantically correct.

<**symbol name**, **type**, **attribute**>

For example, suppose a variable store the information about the following variable declaration:

static int salary ;

then, it stores an entry in the following format: <salary, **int**, **static**>

The clause attribute contains the entries related to the name.

Implementation

The symbol table can be implemented in the unordered list if the compiler is used to handle the small amount of data.

A symbol table can be implemented in one of the following techniques:

- Linear (sorted or unsorted) list
- Hash table
- Binary search tree

Symbol table are mostly implemented as hash table.

Operations of Symbol table – The basic operations defined on a symbol table include:

Operation	Function
allocate	to allocate a new empty symbol table
free	to remove all entries and free storage of symbol table
lookup	to search for a name and return pointer to its entry
insert	to insert a name in a symbol table and return a pointer to its entry
set_attribute	to associate an attribute with a given entry
get_attribute	to get an attribute associated with a given entry

Representing Scope Information

In the source program, every name possesses a region of validity, called **the scope of that name**.

The rules in a block-structured language are as follows:

1. If a name declared within block B then it will be valid only within B.
2. If **B1** block is nested within **B2** then the name that is valid for block **B2** is also valid for **B1** unless the name's identifier is re-declared in B1.

    - These scope rules need a more complicated organization of symbol table than a list of associations between names and attributes.
    - Tables are organized into stack and each table contains the list of names and their associated attributes.
    - Whenever a new block is entered then a new table is entered onto the stack. The new table holds the name that is declared as local to this block.
    - When the declaration is compiled then the table is searched for a name.
    - If the name is not found in the table then the new name is inserted.
    - When the name's reference is translated then each table is searched, starting from the each table on the stack.

For example:

```
1.  ….
2.  int x;
3.  void f(int m) {
4.      float x, y;
5.  {
6.          int i, j;
7.          int u, v;
8.  }
9.  }
10. int g (int n)
11. {
12.     bool f;
13. }
```

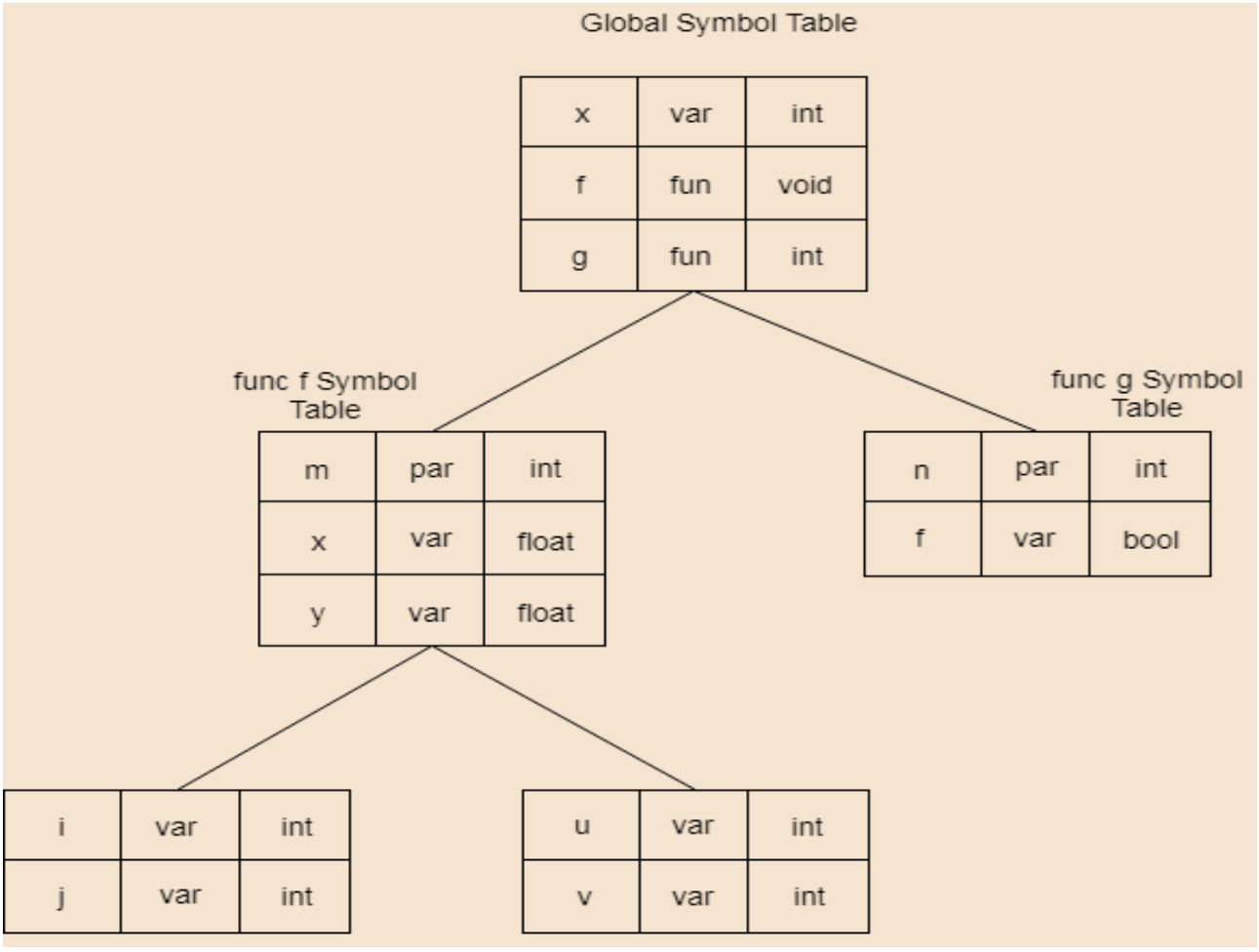


Fig: Symbol table organization that complies with static scope information rules